

# From Zero to the Metal

A Self-Directed Project Roadmap Leading to  
Programming Language Theory · Compilers · Operating System Kernels

This roadmap is designed to take you from foundational systems programming all the way to implementing a compiler with a custom IR and writing a minimal operating system kernel — entirely through hands-on projects. Each phase builds directly on the last. Languages used are C, Rust, and eventually a small amount of Assembly. All work assumes a Linux environment. Every project is intended to be version-controlled on GitHub as a public portfolio demonstrating genuine depth, not tutorial-following.

|          |              |                |              |            |
|----------|--------------|----------------|--------------|------------|
| 8 Phases | 40+ Projects | C · Rust · ASM | Linux Native | ~2–4 Years |
|----------|--------------|----------------|--------------|------------|

## Contents

### Philosophy & How to Use This Document

**Phase 1** — Linux Foundations & C Basics

**Phase 2** — C Mastery — Memory, Data Structures, Systems Calls

**Phase 3** — Rust Foundations & Systems Programming

**Phase 4** — Formal Grounding — Discrete Math, Logic & Automata

**Phase 5** — Interpreter & Virtual Machine Construction

**Phase 6** — Compiler Frontend — Lexing, Parsing, Semantic Analysis

**Phase 7** — Compiler Backend — IR, Optimization, Code Generation

**Phase 8** — Operating System Kernel Construction

**Supplementary Track A** — Assembly & Low-Level Architecture

**Supplementary Track B** — Type Theory & PL Theory Deep Dive

**Supplementary Track C** — Toolchain & Developer Environment Mastery

**Reading List** — Essential Books by Phase

**GitHub Strategy** — Repository Layout & Commit Discipline

# Philosophy & How to Use This Document

---

## Build, Don't Read

Every resource listed here is secondary to the project itself. Books and papers exist to unblock you when you are stuck, not to be completed before you start. Open a terminal, create a repo, and write code. The roadmap is intentionally project-first.

## Struggle Is the Curriculum

These projects are hard by design. When a linker error or a segfault stops you for three days, that is not failure — that is the actual lesson. Keep a dev log in each repo's DEVLOG.md. Writing down what broke and why is worth more than any tutorial.

## Compilers and Kernels Are Not Magic

They are large C and Rust programs with well-understood architecture. The goal of this roadmap is to demystify them incrementally. By the time you write a kernel, you will have already written a heap allocator, a parser, a bytecode VM, a register allocator — each in isolation. The kernel is just the integration.

## Language Choice is Deliberate

C is used first because the kernel and most existing systems are written in C. You need to be able to read and reason about C fluently before touching a kernel. Rust is introduced in Phase 3 and becomes the primary language for the compiler and kernel work, because its ownership model makes systems programming tractable without a garbage collector. Assembly is introduced as a supplementary track and woven into later phases as needed.

## Linux is the Environment

All projects assume Linux. Use whatever distribution suits you — Arch, Debian, Fedora. The kernel projects will require either bare-metal or QEMU/x86\_64. Familiarity with the Linux toolchain (gcc, clang, gdb, objdump, strace, make, cmake, ld) is a first-class outcome of this roadmap, not a prerequisite.

## Pacing

Phases 1–3 can be done in parallel with heavy day-job schedules — 30–60 minutes a day is enough to make progress. Phases 5–8 will require longer focused sessions. There is no deadline. Depth over speed, always.

Before writing a parser or a kernel, you need to be comfortable at the command line and in C. This phase is about establishing fluency, not mastery. You are building the muscle memory and mental models that everything else depends on. Expect to spend a few weeks here. Do not rush it.

### ◆ Project 1-A: Linux Environment Setup & Shell Mastery

*Lang: Bash / Shell Difficulty: Beginner Est. time: 1–2 weeks*

#### Goals

- Configure a productive Linux development environment from scratch
- Achieve fluency with the tools you will use every day for the rest of this roadmap
- Understand the filesystem hierarchy, permissions model, and process model

#### Tasks

1. Install Linux on bare metal or a VM. Configure i3/sway or a minimal WM if preferred.
2. Configure Neovim with LSP support for C and Rust (use lazy.nvim, clangd, rust-analyzer).
3. Write a dotfiles repo. Track your shell config, editor config, aliases. Push to GitHub.
4. Complete the missing semester of CS (MIT OCW) — shell tools, vim, git, tmux modules.
5. Write a shell script that automates your dev environment setup from a fresh install.
6. Learn gdb basics: compile with -g, set breakpoints, step through code, inspect memory.
7. Learn make: write a Makefile from scratch for a simple multi-file C project.

#### Key Resources

- The Missing Semester of Your CS Education — [missing.csail.mit.edu](https://missing.csail.mit.edu)
- tldr-pages (apt install tldr) — fast man page summaries
- Arch Wiki (applicable to any distro for conceptual depth)
- gdb Tutorial — [sourceware.org/gdb/current/onlinedocs/gdb/](https://sourceware.org/gdb/current/onlinedocs/gdb/)

*Note: Your dotfiles repo is your first GitHub artifact. Keep it public and maintained forever.*

## ◆ Project 1-B: C Fundamentals — The K&R; Projects

*Lang: C Difficulty: Beginner–Intermediate Est. time: 3–5 weeks*

### Goals

- Gain fluency in C syntax, pointers, arrays, and basic I/O
- Understand the compilation pipeline: preprocessing, compilation, assembly, linking
- Get comfortable reading compiler output and error messages

### Tasks

1. Read K&R; chapters 1–6. Implement every exercise. No skipping.
2. Project: implement `wc` (word count) — handle `stdin`, files, flags `-l -w -c`.
3. Project: implement `cat` with `-n` (line numbers) and `-A` (show non-printing chars).
4. Project: implement a basic calculator that parses infix expressions via a loop.
5. Write a Makefile for each project. Add a clean target and a test target.
6. Use `valgrind` on every project. Fix every leak and invalid access before moving on.
7. Read compiler output: use `gcc -E` to inspect preprocessor, `gcc -S` to inspect assembly.

### Key Resources

- The C Programming Language — Kernighan & Ritchie (2nd ed.) — mandatory
- Beej's Guide to C Programming — [beej.us/guide/bgc/](http://beej.us/guide/bgc/)
- `valgrind` manual — [valgrind.org/docs/manual/manual.html](http://valgrind.org/docs/manual/manual.html)
- [cdecl.org](http://cdecl.org) — decode C type declarations

*Note: K&R; exercises are deliberately terse. If an exercise takes a week, that is fine.*

## ◆ Project 1-C: Git Discipline & GitHub Portfolio Initialization

*Lang: Git / Markdown Difficulty: Beginner Est. time: 1 week (ongoing)*

### Goals

- Establish version control habits that will serve every project on this roadmap
- Initialize a GitHub profile that reflects real work, not tutorial clones

### Tasks

1. Create a GitHub profile README.md. Keep it minimal and honest.
2. Learn git internals: objects, trees, commits, refs. Read 'Git from the Bottom Up'.
3. Practice rebasing, cherry-picking, and resolving merge conflicts intentionally.
4. Write a CONTRIBUTING.md and README.md template you will reuse across projects.
5. Set up SSH key auth, signing commits with GPG (optional but professional).

### Key Resources

- Pro Git — [git-scm.com/book](http://git-scm.com/book) (free online, read chapters 1–3 and 10)
- Git from the Bottom Up — [jwiegley.github.io/git-from-the-bottom-up/](http://jwiegley.github.io/git-from-the-bottom-up/)

This phase is where C gets serious. You will build your own implementations of fundamental data structures, a memory allocator, and programs that interact directly with the Linux kernel via system calls. By the end of this phase, memory layout, stack frames, heap allocation, and file descriptors should feel like home.

### ◆ Project 2-A: Data Structures Library in C

*Lang: C Difficulty: Intermediate Est. time: 4–6 weeks*

#### Goals

- Implement common data structures from scratch with clean APIs
- Deeply understand pointer arithmetic and memory ownership
- Practice writing tests in C (use the Unity test framework)

#### Tasks

1. Implement: singly linked list, doubly linked list, dynamic array (vec), hash map (open addressing), binary search tree, min-heap.
2. Each structure must have a header (.h), implementation (.c), and test file.
3. Support generic void\* payloads. Add iterator patterns where applicable.
4. Implement a simple arena allocator for the hash map to avoid fragmentation.
5. Write a benchmark harness: compare your hash map vs glibc's hsearch.
6. Document time/space complexity for every operation in the README.

#### Key Resources

- Mastering Algorithms with C — Kyle Loudon
- Introduction to Algorithms (CLRS) — for complexity reference, not cover-to-cover
- Unity test framework — [github.com/ThrowTheSwitch/Unity](https://github.com/ThrowTheSwitch/Unity)

*Note: This repo becomes a library you link against in later projects.*

## ◆ Project 2-B: Write a Memory Allocator (malloc/free)

*Lang: C Difficulty: Intermediate–Hard Est. time: 3–4 weeks*

### Goals

- Understand how the heap is managed at the OS level
- Implement first-fit, best-fit, and free-list coalescing strategies
- Understand sbrk/mmap and when each is appropriate

### Tasks

1. Implement a drop-in malloc replacement using sbrk(). Export as a shared library.
2. Add a block header/footer scheme for boundary-tag coalescing.
3. Implement free list management with explicit free list (not implicit).
4. Add a realloc() that avoids copying when possible.
5. Test with LD\_PRELOAD=./mymalloc.so on real programs. Debug with valgrind.
6. Read the glibc ptmalloc source for one allocator bin size. Write notes on what you find.
7. Stretch: add thread safety with a mutex and test with concurrent workloads.

### Key Resources

- CS:APP Chapter 9 — Virtual Memory (Bryant & O'Hallaron)
- Writing a Memory Allocator — [dmitrysoshnikov.com/compilers/writing-a-memory-allocator/](https://dmitrysoshnikov.com/compilers/writing-a-memory-allocator/)
- glibc malloc source — [sourceware.org/git/glibc.git](https://sourceware.org/git/glibc.git)

## ◆ Project 2-C: Systems Programming — Shell Implementation

*Lang: C Difficulty: Intermediate–Hard Est. time: 4–6 weeks*

### Goals

- Understand fork/exec/wait, file descriptors, pipes, and signals
- Implement job control (foreground/background, Ctrl-C, Ctrl-Z)
- Read and understand the POSIX specification for process management

### Tasks

1. Implement a basic REPL that parses and executes commands.
2. Add I/O redirection: `>`, `>>`, `<`, `2>`, `&>`.
3. Add piping: `cmd1 | cmd2 | cmd3` with proper fd management.
4. Implement built-ins: `cd`, `exit`, `export`, `unset`, `jobs`, `fg`, `bg`.
5. Implement job control: SIGCHLD handling, process groups, `tcsetpgrp`.
6. Add a simple command history (up-arrow) without readline dependency.
7. Write integration tests using Python's subprocess module.

### Key Resources

- Advanced Programming in the UNIX Environment — Stevens & Rago (APUE)
- The Linux Programming Interface — Kerrisk (reference bible)
- POSIX.1-2017 — [pubs.opengroup.org/onlinepubs/9699919799/](https://pubs.opengroup.org/onlinepubs/9699919799/)
- Stanford CS110 shell assignment — public writeup available online

*Note: This is one of the hardest projects in Phase 2. Signal handling especially. Take notes.*

## ◆ Project 2-D: Network Client/Server in C

*Lang: C Difficulty: Intermediate Est. time: 2–3 weeks*

### Goals

- Understand BSD sockets API, TCP connection lifecycle
- Implement a simple HTTP/1.0 server that serves static files

### Tasks

1. Write a TCP echo server and client. Handle multiple clients with `select()` or `poll()`.
2. Implement an HTTP/1.0 server: parse GET requests, serve files, return proper status codes.
3. Add support for MIME types and basic error pages (404, 403).
4. Stress-test with curl and Apache Bench (ab). Find and fix your bottlenecks.
5. Read RFC 7230 (HTTP/1.1 message syntax) for one afternoon. Not the whole thing.

### Key Resources

- Beej's Guide to Network Programming — [beej.us/guide/bgnet/](http://beej.us/guide/bgnet/)
- The Linux Programming Interface — chapters 56–61
- RFC 7230 — [datatracker.ietf.org/doc/html/rfc7230](https://datatracker.ietf.org/doc/html/rfc7230)

Rust is introduced here because you now understand what it is protecting you from. Having written a memory allocator and a shell in C, ownership and lifetimes will make intuitive sense rather than feeling like arbitrary restrictions. The projects in this phase mirror Phase 2 in spirit but use Rust idioms — iterators, traits, error handling with Result, and the borrow checker.

### ◆ Project 3-A: Rust Language Fundamentals

*Lang: Rust Difficulty: Beginner–Intermediate Est. time: 3–4 weeks*

#### Goals

- Achieve fluency with Rust syntax, ownership, and the type system
- Understand traits, generics, lifetimes, and closures
- Use cargo, clippy, rustfmt, and rust-analyzer in your daily workflow

#### Tasks

1. Read The Rust Programming Language (the Book) chapters 1–15. Do every exercise.
2. Complete Rustlings ([github.com/rust-lang/rustlings](https://github.com/rust-lang/rustlings)) — all exercises.
3. Port your C linked list to Rust. Use unsafe only where forced. Document why.
4. Implement a generic stack and queue using Rust's Vec and VecDeque.
5. Write a CLI tool with clap that wraps a system command and adds flags.
6. Add unit tests (`#[test]`) and integration tests (`/tests/`) for every module.

#### Key Resources

- The Rust Programming Language — [doc.rust-lang.org/book/](https://doc.rust-lang.org/book/) (free)
- Rustlings — [github.com/rust-lang/rustlings](https://github.com/rust-lang/rustlings)
- Rust by Example — [doc.rust-lang.org/rust-by-example/](https://doc.rust-lang.org/rust-by-example/)
- Jon Gjengset's 'Crust of Rust' series — YouTube (excellent for intermediate gaps)



## ◆ Project 3-B: Rust Data Structures & Algorithms

*Lang: Rust Difficulty: Intermediate Est. time: 3–4 weeks*

### Goals

- Implement idiomatic Rust data structures using generics and traits
- Understand the limits of safe Rust for intrusive data structures

### Tasks

1. Implement a generic hash map using open addressing and Robin Hood hashing.
2. Implement a B-Tree (order 4) with insert, delete, and range scan.
3. Implement a lock-free MPSC queue using `Arc<` then with atomics.
4. Port your C arena allocator to Rust using `unsafe`. Document every `unsafe` block.
5. Write benchmarks with `criterion.rs` comparing your structures to std equivalents.

### Key Resources

- Learn Rust With Entirely Too Many Linked Lists — [rust-unofficial.github.io/too-many-lists/](https://rust-unofficial.github.io/too-many-lists/)
- The Rustonomicon — [doc.rust-lang.org/nomicon/](https://doc.rust-lang.org/nomicon/) (for `unsafe` Rust)
- `criterion.rs` — [bheisler.github.io/criterion.rs/book/](https://bheisler.github.io/criterion.rs/book/)

## ◆ Project 3-C: Systems Programming in Rust

*Lang: Rust Difficulty: Intermediate–Hard Est. time: 4–5 weeks*

### Goals

- Use Rust's `nix` and `libc` crates for POSIX system call access
- Understand where Rust's safety model requires `unsafe` boundaries at the OS interface

### Tasks

1. Implement a minimal shell in Rust using `nix` crate (mirror your C shell).
2. Implement a multi-threaded HTTP server using only `std::net` and `std::thread`.
3. Implement a file watcher using `inotify` (Linux) via `nix` — trigger on write events.
4. Build a process monitor (like `htop` lite): read `/proc/[pid]/stat`, display sorted table.
5. Write a toy key-value store with a WAL (write-ahead log) persisted to disk.

### Key Resources

- `nix` crate documentation — [docs.rs/nix](https://docs.rs/nix)
- Linux `/proc` filesystem — [kernel.org/doc/html/latest/filesystems/proc.html](https://kernel.org/doc/html/latest/filesystems/proc.html)
- Rust Systems Programming — O'Reilly (Jim Blandy et al.)

This phase is the academic interlude. You do not need to become a mathematician, but compilers and type systems are built on formal foundations that will make you dramatically more effective if you have encountered them. The goal is working familiarity, implemented through code — not proof-writing for its own sake.

### ◆ Project 4-A: Finite Automata & Regular Expressions Engine

*Lang: Rust Difficulty: Intermediate Est. time: 3–4 weeks*

#### Goals

- Understand NFAs, DFAs, and the Thompson construction
- Implement regex to NFA to DFA compilation
- Understand the formal definition of a regular language

#### Tasks

1. Implement Thompson's construction: convert a regex AST to an NFA.
2. Implement subset construction: convert NFA to DFA.
3. Implement DFA minimization (Hopcroft's algorithm).
4. Build a simple regex engine: support `.`, `*`, `+`, `?`, `|`, character classes `[]`.
5. Write a test suite using known regex edge cases. Compare output to Python's `re` module.
6. Write a README explaining the theory behind each transformation.

#### Key Resources

- Introduction to the Theory of Computation — Sipser (chapters 1–2)
- Compilers: Principles, Techniques & Tools (Dragon Book) — chapter 3
- Regular Expression Matching Can Be Simple And Fast — [rsc.io/regexp/syntax](https://rsc.io/regexp/syntax)

## ◆ Project 4-B: Context-Free Grammars & Parsing Theory

*Lang: Rust Difficulty: Intermediate–Hard Est. time: 3–4 weeks*

### Goals

- Understand context-free grammars, parse trees, and ambiguity
- Implement LL(1) and LR(0) parsers from scratch
- Understand FIRST, FOLLOW, and item set construction

### Tasks

1. Implement an LL(1) parser for a simple expression grammar. Build parse table by hand.
2. Implement an LR(0) parser: build the canonical collection of item sets, parse table.
3. Implement a Pratt parser (top-down operator precedence) for arithmetic expressions.
4. Implement an Earley parser for an ambiguous grammar.
5. Write a tool that takes a grammar file (BNF notation) and generates FIRST/FOLLOW sets.

### Key Resources

- Crafting Interpreters — Robert Nystrom — [craftinginterpreters.com](https://craftinginterpreters.com) (free online)
- Engineering a Compiler — Cooper & Torczon (chapters 3–5)
- Simple but Powerful Pratt Parsing — [matklad.github.io](https://matklad.github.io)

## ◆ Project 4-C: Logic, Type Systems & Lambda Calculus

*Lang: Rust / Haskell (optional) Difficulty: Hard Est. time: 4–6 weeks*

### Goals

- Understand propositional and predicate logic at a working level
- Implement the untyped lambda calculus with beta reduction
- Implement simply typed lambda calculus with Hindley-Milner type inference

### Tasks

1. Implement a lambda calculus evaluator: parse, substitute, beta-reduce.
2. Add de Bruijn indices to avoid capture-avoiding substitution bugs.
3. Implement Algorithm W (Hindley-Milner) for type inference.
4. Add let-polymorphism. Test with Church numerals and list encodings.
5. Write a paper-length README explaining your implementation and the theory behind it.
6. Stretch: add simple sum types (enums) and pattern matching to your type checker.

### Key Resources

- Types and Programming Languages — Pierce (TAPL) — the canonical reference
- Programming Language Foundations in Agda — [plfa.inf.ed.ac.uk](https://plfa.inf.ed.ac.uk) (free online)
- Write You a Haskell — [stephendiehl.com/fun/](https://stephendiehl.com/fun/)
- Lectures on the Curry-Howard Correspondence — Philip Wadler

*Note: TAPL is dense but worth it. Read chapters 3, 8, 9, 22–23 at minimum for this project.*

This is where the compiler pipeline becomes concrete. You will build a complete tree-walking interpreter and then a bytecode virtual machine for a language you design. These are the most creatively satisfying projects on the roadmap and the ones that make everything in later phases click into place.

### ◆ Project 5-A: Tree-Walking Interpreter for a Toy Language

*Lang: Rust Difficulty: Intermediate–Hard Est. time: 6–8 weeks*

#### Goals

- Implement a complete language pipeline: lexer → parser → AST → evaluator
- Understand environments, closures, and first-class functions
- Design a language with a real grammar, not just arithmetic

#### Tasks

1. Design a language: at minimum support integers, booleans, strings, functions, closures, if/else, while, let.
2. Write a hand-rolled lexer (no lex/flex). Represent tokens as an enum.
3. Write a recursive descent parser producing a typed AST enum.
4. Write a tree-walking evaluator using an environment chain for scoping.
5. Add a REPL with line editing.
6. Write an extensive test suite covering edge cases in scoping and closures.
7. Stretch: add first-class arrays, hash maps, and a small standard library.

#### Key Resources

- Writing An Interpreter In Go — Thorsten Ball (conceptually translate to Rust)
- Crafting Interpreters Part I & II — [craftinginterpreters.com](http://craftinginterpreters.com)
- Crafting Interpreters in Rust — [shadyfennec.github.io](http://shadyfennec.github.io) (community translation)

*Note: Do NOT use parser combinators (nom, pest) here. The goal is to understand the mechanics.*

## ◆ Project 5-B: Bytecode Compiler & Stack-Based VM

*Lang: Rust Difficulty: Hard Est. time: 6–8 weeks*

### Goals

- Understand bytecode instruction sets, operand stacks, and dispatch loops
- Implement a mark-and-sweep garbage collector
- Understand how closures are implemented at the bytecode level (upvalues)

### Tasks

1. Design a bytecode instruction set for your language from Project 5-A.
2. Implement a compiler pass from AST to bytecode.
3. Implement a stack-based VM with a dispatch loop (match or computed goto via unsafe).
4. Implement a mark-and-sweep GC. Track allocations. Trigger GC on threshold.
5. Implement upvalues for closures (Lua-style: open/close upvalue lifecycle).
6. Add a disassembler that pretty-prints bytecode for debugging.
7. Benchmark your VM vs your tree-walker on a fibonacci(35) workload.

### Key Resources

- Writing A Compiler In Go — Thorsten Ball
- Crafting Interpreters Part III — [craftinginterpreters.com](http://craftinginterpreters.com) (clox is the reference)
- A Tour of V8: Garbage Collection — [jayconrod.com](http://jayconrod.com)

## ◆ Project 5-C: Register-Based VM

*Lang: Rust Difficulty: Hard Est. time: 4–5 weeks*

### Goals

- Understand the difference between stack-based and register-based VMs
- Implement a register allocator for a simple IR

### Tasks

1. Design a register-based bytecode (Lua 5's VM is a good reference).
2. Rewrite your compiler backend to emit register bytecode.
3. Implement a simple linear scan register allocator.
4. Benchmark stack vs register VM on the same workloads.
5. Write a README analyzing the tradeoffs with measured data.

### Key Resources

- The Implementation of Lua 5.0 — [roberto.lua.org/doc/jucs05.pdf](http://roberto.lua.org/doc/jucs05.pdf)
- Linear Scan Register Allocation — Poletto & Sarkar 1999 (find PDF via Semantic Scholar)

The projects in this phase are compiler-quality, not interpreter-quality. You are now building the front half of a real compiler: one that handles a statically typed language, performs full semantic analysis, produces a typed AST, and reports meaningful errors. This is where type theory from Phase 4 becomes engineering.

### ◆ Project 6-A: Lexer Generator

*Lang: Rust Difficulty: Intermediate–Hard Est. time: 3–4 weeks*

#### Goals

- Build a tool that takes a lexer specification and generates a Rust lexer
- Understand DFA simulation and the performance tradeoffs of lexer generation

#### Tasks

1. Define a lexer spec format (regex rules + token names), similar to lex/flex.
2. Parse the spec, build NFAs per rule, combine, convert to DFA.
3. Generate Rust source code implementing the DFA as a state transition table.
4. Handle priority rules (longer match wins; earlier rule wins on tie).
5. Test on your own language grammar from Phase 5.
6. Compare performance of generated DFA lexer vs hand-rolled lexer.

#### Key Resources

- Dragon Book Chapter 3 — Lexical Analysis
- Engineering a Compiler Chapter 2 — Cooper & Torczon
- flex source code — [github.com/westes/flex](https://github.com/westes/flex) (for reference, not copying)

## ◆ Project 6-B: Parser Generator (LALR(1))

*Lang: Rust Difficulty: Hard–Very Hard Est. time: 6–8 weeks*

### Goals

- Understand LALR(1) item set construction and shift-reduce conflict resolution
- Build a tool that takes a grammar and generates a parse table
- Generate Rust source code for a parser

### Tasks

1. Implement LR(1) canonical collection construction.
2. Implement LALR(1) merging from LR(1) item sets.
3. Build shift-reduce and reduce-reduce conflict reporting.
4. Generate a parse table (action/goto tables) and a Rust parser skeleton.
5. Test on your own language grammar. Resolve all conflicts.
6. Stretch: implement operator precedence declarations to resolve ambiguity.

### Key Resources

- Engineering a Compiler Chapters 3–5
- Dragon Book Chapters 4
- bison source — [gnu.org/software/bison/](http://gnu.org/software/bison/) (reference, not copying)
- LALR(1) Grammars — Stanford CS143 lecture notes (public)

*Note: This is genuinely hard. Do not be discouraged if it takes twice the estimated time.*

## ◆ Project 6-C: Semantic Analysis & Type Checker for a Statically Typed Language

*Lang: Rust Difficulty: Hard Est. time: 5–7 weeks*

### Goals

- Implement symbol table management, scope resolution, and name binding
- Implement type checking for a statically typed, expression-oriented language
- Produce useful, located error messages

### Tasks

1. Define a statically typed language: add type annotations, structs, enums, generics.
2. Implement symbol table with lexical scoping (use a scope stack).
3. Implement type inference via bidirectional type checking or Algorithm W.
4. Add exhaustiveness checking for pattern matching.
5. Implement a typed AST (HIR) that carries type information on every node.
6. Write a dedicated error reporting module with source spans and hints.
7. Write a test suite of programs that should pass and programs that should fail with specific errors.

### Key Resources

- Types and Programming Languages — Pierce (chapters 11–15)
- rustc\_hir source — [github.com/rust-lang/rust](https://github.com/rust-lang/rust) (for inspiration on HIR design)
- Bidirectional Typing — David Christiansen lecture notes

The backend is where the compiler earns its keep. You will implement your own intermediate representation in SSA form, run optimization passes, and generate x86-64 assembly. Alternatively, you can target LLVM IR as a backend, which is the pragmatic production path. Doing both teaches the most.

### ◆ Project 7-A: Intermediate Representation in SSA Form

*Lang: Rust Difficulty: Hard Est. time: 4–6 weeks*

#### Goals

- Design a three-address code IR
- Implement SSA construction via dominance frontiers
- Understand phi nodes and their role in optimization

#### Tasks

1. Define an IR: basic blocks, instructions (load, store, add, cmp, branch, call, phi).
2. Implement IR construction from your typed AST.
3. Implement dominance tree construction (Lengauer-Tarjan algorithm).
4. Implement SSA construction via dominance frontier pruned insertion.
5. Implement SSA deconstruction (phi elimination) for code generation.
6. Write an IR pretty-printer and verifier that checks SSA invariants.

#### Key Resources

- Engineering a Compiler Chapters 8–9
- SSA-based Compiler Design — Braun et al. (free PDF on Semantic Scholar)
- Simple and Efficient Construction of SSA — Braun et al. 2013



## ◆ Project 7-B: Optimization Passes

*Lang: Rust Difficulty: Hard Est. time: 4–6 weeks*

### Goals

- Implement classical dataflow-based optimizations
- Understand the pass manager pattern used in real compilers

### Tasks

1. Implement constant folding and constant propagation.
2. Implement dead code elimination.
3. Implement common subexpression elimination (CSE) using value numbering.
4. Implement inlining for small functions.
5. Implement loop-invariant code motion (LICM).
6. Design a pass manager that runs passes to fixed point.
7. Measure compile time and runtime improvement with and without each pass.

### Key Resources

- Engineering a Compiler Chapters 8–10
- Modern Compiler Implementation in ML — Appel (algorithms section)
- LLVM's pass documentation — [llvm.org/docs/Passes.html](http://llvm.org/docs/Passes.html) (for comparison)

## ◆ Project 7-C: x86-64 Code Generation

*Lang: Rust + x86-64 ASM Difficulty: Very Hard Est. time: 6–8 weeks*

### Goals

- Generate valid x86-64 assembly from your IR
- Implement instruction selection and register allocation
- Produce a working ELF binary via an assembler

### Tasks

1. Implement instruction selection via tree pattern matching (IBURG-style or ad hoc).
2. Implement linear scan register allocation for x86-64 (RAX, RBX, RCX, RDX, RSI, RDI, R8–R15).
3. Handle spilling: generate load/store to stack slots.
4. Emit x86-64 AT&T; syntax assembly. Use GAS (as) to assemble.
5. Implement the System V AMD64 ABI for function calls.
6. Link with ld against your runtime (write a tiny runtime.c for malloc, print, exit).
7. Compile a non-trivial program end-to-end. Verify with objdump -d.

### Key Resources

- x86-64 Assembly Language Programming with Ubuntu — Ed Jorgensen (free PDF)
- System V Application Binary Interface — [refspecs.linuxfoundation.org](http://refspecs.linuxfoundation.org)
- Intel Software Developer's Manual Volume 2 (instruction reference)
- Engineering a Compiler Chapter 11

*Note: This project will take longer than any estimate. Plan for it.*

## ◆ Project 7-D: LLVM Backend (Alternative / Parallel Path)

*Lang: Rust + LLVM IR Difficulty: Hard Est. time: 3–5 weeks*

### Goals

- Target LLVM IR as a compilation target
- Understand LLVM's optimization pipeline and how to drive it

### Tasks

1. Use inkwell (Rust LLVM bindings) to emit LLVM IR from your typed AST.
2. Drive clang/opt to run LLVM optimization passes on your IR.
3. Compare LLVM-optimized output vs your hand-rolled optimizer output.
4. Add LLVM's debug info (DWARF) so gdb can step through your compiled programs.
5. Implement a simple calling convention consistent with C for interop.

### Key Resources

- inkwell — [github.com/TheDan64/inkwell](https://github.com/TheDan64/inkwell)
- LLVM Language Reference Manual — [llvm.org/docs/LangRef.html](http://llvm.org/docs/LangRef.html)
- Mapping High Level Constructs to LLVM IR — [llvm.org/docs/tutorial/](http://llvm.org/docs/tutorial/)

This is the summit. By this point you have a thorough understanding of C, Rust, assembly, memory management, and systems programming. The kernel projects here build incrementally — starting with bare-metal bootstrapping and ending with a kernel that can run a shell and support multiple processes. All kernel work runs on QEMU unless you have hardware you are willing to experiment on.

### ◆ Project 8-A: Bare-Metal Bootstrapping & x86-64 Environment

*Lang: C + x86-64 ASM Difficulty: Hard Est. time: 3–4 weeks*

#### Goals

- Boot a kernel to a known state without an OS beneath it
- Initialize the GDT, IDT, and basic interrupt handling

#### Tasks

1. Write a multiboot2-compliant bootloader stub in assembly.
2. Set up a GDT with ring 0 code/data segments.
3. Set up an IDT and handle divide-by-zero and general protection faults.
4. Implement a VGA text mode driver for output.
5. Boot in QEMU with `-kernel`. Verify with QEMU's monitor commands.
6. Set up serial output (COM1) for logging — useful for debugging.

#### Key Resources

- OSDev Wiki — [wiki.osdev.org](http://wiki.osdev.org) (primary reference for all kernel work)
- Writing a Simple Operating System from Scratch — Nick Blundell (free PDF)
- x86-64 Assembly Language Programming with Ubuntu — Jorgensen

## ◆ Project 8-B: Physical & Virtual Memory Management

*Lang: C + ASM Difficulty: Very Hard Est. time: 5–7 weeks*

### Goals

- Understand the x86-64 paging hierarchy (PML4, PDPT, PD, PT)
- Implement a physical memory manager (frame allocator)
- Implement a virtual memory mapper and kernel heap

### Tasks

1. Parse the multiboot2 memory map. Build a bitmap frame allocator.
2. Implement identity mapping for the first 4MB.
3. Implement a 4-level page table walker. Map arbitrary virtual to physical addresses.
4. Implement a kernel heap allocator (use your malloc from Phase 2 as a model).
5. Handle page faults: demand paging stub.
6. Implement a slab allocator for fixed-size kernel objects.

### Key Resources

- OSDev Wiki — Memory Management
- Intel Manual Volume 3A — System Programming Guide (paging chapters)
- Three Easy Pieces — [remzi.cs.wisc.edu/OSTEP/](http://remzi.cs.wisc.edu/OSTEP/) (free online, virtual memory chapters)

## ◆ Project 8-C: Process Scheduler & Context Switching

*Lang: C + ASM Difficulty: Very Hard Est. time: 5–7 weeks*

### Goals

- Implement a preemptive round-robin scheduler using the APIC timer
- Understand context switching at the register/stack level
- Support kernel threads and user-space process isolation

### Tasks

1. Implement a process control block (PCB) with register save area.
2. Implement context switching in assembly (save/restore all callee-saved registers).
3. Initialize the APIC timer. Handle timer interrupts to preempt tasks.
4. Implement a run queue. Schedule with round-robin.
5. Add user mode: use IRET to drop to ring 3. Implement syscall entry (SYSCALL/SYSRET).
6. Implement `sys_write` and `sys_exit` as your first system calls.
7. Implement `sys_fork` and `sys_execve` (even a minimal version).

### Key Resources

- OSDev Wiki — Scheduling, Context Switching
- Operating Systems: Three Easy Pieces — Chapters 4–9 (processes, scheduling)
- xv6-riscv source — [github.com/mit-pdos/xv6-riscv](https://github.com/mit-pdos/xv6-riscv) (best kernel to read alongside)

*Note: Read the xv6 source obsessively alongside this. It is the best kernel learning resource that exists.*

## ◆ Project 8-D: Filesystem & Storage

*Lang: C Difficulty: Hard Est. time: 4–6 weeks*

### Goals

- Implement a simple block device driver and VFS abstraction
- Implement a FAT32 or ext2 filesystem driver

### Tasks

1. Implement an ATA PIO driver for reading sectors from a QEMU disk image.
2. Design a VFS layer: mount points, inodes, file descriptors, read/write ops.
3. Implement a FAT32 driver (simpler) or ext2 driver (more educational).
4. Add `sys_open`, `sys_read`, `sys_write`, `sys_close`, `sys_lseek`.
5. Implement a `/proc`-like virtual filesystem for exposing kernel state.
6. Write a userspace program that reads, writes, and lists files.

### Key Resources

- OSDev Wiki — Filesystems, ATA/ATAPI
- FAT32 Specification — [microsoft.com/en-us/download/details.aspx?id=67584](https://microsoft.com/en-us/download/details.aspx?id=67584)
- ext2 Specification — [nongnu.org/ext2-doc/ext2.html](https://nongnu.org/ext2-doc/ext2.html)

## ◆ Project 8-E: Rust Kernel (Rust-OSDev Path)

*Lang: Rust Difficulty: Very Hard Est. time: 8–12 weeks*

### Goals

- Implement a minimal kernel in Rust using the `no_std` ecosystem
- Understand where Rust's safety model intersects with bare-metal programming

### Tasks

1. Set up a `no_std` Rust project with a custom target spec for `x86-64-unknown-none`.
2. Port your bootloader stub. Use the `x86_64` crate for register access.
3. Port your frame allocator and page table mapper using safe abstractions where possible.
4. Implement a minimal allocator using the `GlobalAlloc` trait.
5. Implement `async/await` for cooperative kernel tasks using a simple executor.
6. Implement a basic driver model using traits.
7. Write a kernel module that tests your entire stack end to end.

### Key Resources

- Writing an OS in Rust — [os.phil-opp.com](https://os.phil-opp.com) (Philipp Oppermann — the canonical resource)
- `x86_64` crate — [docs.rs/x86\\_64](https://docs.rs/x86_64)
- rust-osdev organization — [github.com/rust-osdev](https://github.com/rust-osdev)
- Redox OS source — [gitlab.redox-os.org](https://gitlab.redox-os.org) (real Rust OS for reference)

*Note: Phil Oppermann's blog is extraordinarily well-written. Work through it actively, not passively.*

# Supplementary Track A: Assembly & Low-Level Architecture

Assembly is woven into Phase 8 but benefits from dedicated study earlier. These projects can be started any time after Phase 2.

## ◆ ASM-1: x86-64 Assembly Fundamentals

*Lang: x86-64 ASM (NASM/GAS) Difficulty: Intermediate Est. time: 2–3 weeks*

### Goals

- Read and write x86-64 assembly fluently enough to debug compiler output
- Understand the ABI, calling conventions, and stack frame layout

### Tasks

1. Install NASM. Write and run 'Hello, World' from scratch with `sys_write` syscall.
2. Implement: recursive fibonacci, iterative factorial, string length — all in ASM.
3. Write a C function and inspect its prologue/epilogue with `objdump -d`.
4. Call a C function from ASM and an ASM function from C. Understand parameter passing.
5. Implement a trivial SIMD operation using SSE/AVX intrinsics.
6. Read generated assembly from your Phase 7 compiler and verify correctness.

### Key Resources

- x86-64 Assembly Language Programming with Ubuntu — Ed Jorgensen (free PDF)
- Intel® 64 and IA-32 Architectures Software Developer's Manual
- [godbolt.org](http://godbolt.org) — Compiler Explorer (indispensable for comparing C to ASM output)

## ◆ ASM-2: Computer Architecture Self-Study

*Lang: Theory + Simulation Difficulty: Intermediate Est. time: 4–6 weeks*

### Goals

- Understand CPU microarchitecture: pipelines, caches, branch prediction
- Understand how hardware constraints affect software design decisions

### Tasks

1. Study cache hierarchy: measure L1/L2/L3 latency on your machine with `perf` and a benchmark.
2. Implement a cache-oblivious matrix multiplication and compare to naive.
3. Read about out-of-order execution and write a benchmark that demonstrates false sharing.
4. Use `perf stat` and `perf record` to profile a program. Interpret the output.
5. Study the MESI protocol and write a document explaining cache coherence.
6. Stretch: implement a RISC-V CPU in a hardware description language (optional Verilog).

### Key Resources

- Computer Organization and Design RISC-V Edition — Patterson & Hennessy
- What Every Programmer Should Know About Memory — Ulrich Drepper (free PDF)
- `perf` wiki — [perf.wiki.kernel.org](http://perf.wiki.kernel.org)

# Supplementary Track B: Type Theory & PL Theory Deep Dive

This track is for after Phase 4 and can be pursued in parallel with Phases 6–8. It is the path toward understanding the theory that underpins ML, Haskell, Rust's type system, and academic PL research.

## ◆ PLT-1: System F and Polymorphism

*Lang: Rust or Haskell Difficulty: Hard Est. time: 4–5 weeks*

### Goals

- Implement System F (polymorphic lambda calculus)
- Understand universal and existential types

### Tasks

1. Extend your STLC implementation from Phase 4 with type abstraction and application.
2. Implement type erasure (produce STLC terms from System F).
3. Encode data types in System F (Church encoding for lists, Option).
4. Prove (informally) parametricity for a simple polymorphic function.
5. Read Reynolds' 'Types, Abstraction, and Parametric Polymorphism' — write a summary.

### Key Resources

- Types and Programming Languages — Pierce (chapters 23–24)
- Theorems for Free — Philip Wadler (classic paper, find on Wadler's site)

## ◆ PLT-2: Dependent Types — Introduction

*Lang: Agda or Idris Difficulty: Very Hard Est. time: 6–8 weeks*

### Goals

- Understand the Curry-Howard correspondence at a practical level
- Write and prove simple theorems in a dependently typed language
- Understand how dependent types enable verified programming

### Tasks

1. Install Agda. Complete the first four chapters of Programming Language Foundations in Agda.
2. Prove: natural number addition is commutative. Prove: list append is associative.
3. Implement a length-indexed vector type. Write a type-safe head that cannot fail.
4. Implement a verified sort: return a proof that the output is sorted and a permutation of the input.
5. Write a README bridging what you implemented to how Rust's type system approaches the same problems.

### Key Resources

- Programming Language Foundations in Agda — [plfa.inf.ed.ac.uk](http://plfa.inf.ed.ac.uk) (free)
- Idris 2 Tutorial — [idris2.readthedocs.io](http://idris2.readthedocs.io)
- The Little Typer — Friedman & Christiansen

### ◆ PLT-3: Operational Semantics & Mechanized Proofs

*Lang: Agda / Coq Difficulty: Very Hard Est. time: 6–10 weeks*

#### Goals

- Write formal operational semantics for your toy language
- Mechanize a type soundness proof (progress + preservation)

#### Tasks

1. Define big-step and small-step operational semantics for your STLC.
2. Prove progress: a well-typed term is either a value or can take a step.
3. Prove preservation: if a term steps, its type is preserved.
4. Prove type safety as a corollary.
5. Write a paper-quality README documenting the proof structure.

#### Key Resources

- Software Foundations Volume 2: Programming Language Foundations — [softwarefoundations.cis.upenn.edu](http://softwarefoundations.cis.upenn.edu) (free)
- Concrete Semantics — Nipkow & Klein (free PDF)

## Supplementary Track C: Toolchain & Developer Environment Mastery

---

These are not glamorous projects, but they pay dividends across everything else. They can be interleaved throughout the roadmap as needed.



## ◆ TOOL-1: Neovim as a Full IDE for C and Rust

*Lang: Lua / Shell Difficulty: Beginner Est. time: 1–2 weeks*

### Goals

- Configure a productive Neovim environment with LSP, debugging, and testing integration

### Tasks

1. Set up lazy.nvim with clangd, rust-analyzer, lua\_ls.
2. Configure nvim-dap with codeLLDB for debugging C and Rust.
3. Configure telescope.nvim for fuzzy finding across files and LSP symbols.
4. Write a custom statusline or configure lualine with meaningful information.
5. Configure conform.nvim for auto-formatting on save (clang-format, rustfmt).
6. Write your own simple Neovim plugin in Lua (even just a keybinding macro).

### Key Resources

- Neovim documentation — [neovim.io/doc/](https://neovim.io/doc/)
- TJ DeVries YouTube channel — Neovim core contributor
- nvim-lspconfig — [github.com/neovim/nvim-lspconfig](https://github.com/neovim/nvim-lspconfig)

## ◆ TOOL-2: Build Systems & Toolchain Deep Dive

*Lang: Make / CMake / Cargo Difficulty: Intermediate Est. time: 2–3 weeks*

### Goals

- Understand make dependency graphs and recursive make pitfalls
- Write a non-trivial CMake project from scratch

### Tasks

1. Read 'Recursive Make Considered Harmful'. Rewrite a recursive Makefile as flat.
2. Port a multi-library C project to CMake with proper target-based includes and linking.
3. Write a custom Cargo build script (build.rs) that links a C library.
4. Set up a CI pipeline with GitHub Actions for C (with valgrind) and Rust projects.
5. Implement a simple Makefile-based test runner for C.

### Key Resources

- CMake documentation — [cmake.org/cmake/help/latest/](https://cmake.org/cmake/help/latest/)
- Recursive Make Considered Harmful — [miller.emu.id.au/pmiller/books/rmch/](https://miller.emu.id.au/pmiller/books/rmch/)
- GitHub Actions documentation — [docs.github.com/en/actions](https://docs.github.com/en/actions)

### ◆ TOOL-3: Debugging & Profiling Toolchain

*Lang: gdb / valgrind / perf Difficulty: Intermediate Est. time: 2 weeks (ongoing)*

#### Goals

- Use gdb effectively for non-trivial debugging sessions
- Use valgrind's memcheck, callgrind, and helgrind tools

#### Tasks

1. Write a .gdbinit with useful macros for printing linked lists and trees.
2. Use valgrind --tool=callgrind and kcache-grind to profile a data structure benchmark.
3. Use perf record / perf report on your VM interpreter. Identify the hot path.
4. Use AddressSanitizer (ASAN) and UndefinedBehaviorSanitizer (UBSAN) on all C projects.
5. Write a gdb Python script that pretty-prints one of your AST types.

#### Key Resources

- GDB documentation — [sourceware.org/gdb/current/onlinedocs/gdb/](http://sourceware.org/gdb/current/onlinedocs/gdb/)
- Valgrind manual — [valgrind.org/docs/manual/](http://valgrind.org/docs/manual/)
- Linux perf Examples — [brendangregg.com/perf.html](http://brendangregg.com/perf.html)

# Essential Reading List by Phase

---

## Phases 1–2 (C & Systems)

- The C Programming Language — Kernighan & Ritchie
- Advanced Programming in the UNIX Environment — Stevens & Rago
- The Linux Programming Interface — Kerrisk
- Computer Systems: A Programmer's Perspective — Bryant & O'Hallaron

## Phase 3 (Rust)

- The Rust Programming Language — Klabnik & Nichols (free online)
- Programming Rust — Blandy, Orendorff & Tindall
- The Rustonomicon — [doc.rust-lang.org/nomicon/](http://doc.rust-lang.org/nomicon/)

## Phase 4 (Theory)

- Introduction to the Theory of Computation — Sipser
- Types and Programming Languages — Pierce (TAPL)
- Programming Language Foundations in Agda — free online

## Phases 5–7 (Compilers & Interpreters)

- Crafting Interpreters — Robert Nystrom (free online)
- Engineering a Compiler — Cooper & Torczon
- Modern Compiler Implementation in ML — Appel
- Compilers: Principles, Techniques & Tools — Aho et al. (Dragon Book)

## Phase 8 (Kernels & OS)

- Operating Systems: Three Easy Pieces — Arpaci-Dusseau (free online)
- Writing an OS in Rust — [os.phil-opp.com](http://os.phil-opp.com) (free online)
- Understanding the Linux Kernel — Bovet & Cesati
- xv6: a simple, Unix-like teaching OS — [pdos.csail.mit.edu/6.828/](http://pdos.csail.mit.edu/6.828/)

## Architecture & Hardware

- Computer Organization and Design RISC-V Edition — Patterson & Hennessy
- What Every Programmer Should Know About Memory — Drepper (free PDF)

## PL Theory Deep Dive

- Practical Foundations of Mathematics — Taylor
- Software Foundations — free online
- Concrete Semantics — Nipkow & Klein (free PDF)
- The Little Typer — Friedman & Christiansen

# GitHub Strategy — Repository Layout & Commit Discipline

---

## Repository Naming Convention

| Repo Name       | Purpose                                                                            |
|-----------------|------------------------------------------------------------------------------------|
| dotfiles        | Your development environment. Always public.                                       |
| clib            | Your Phase 2 C data structures library. Single repo, subdirs per structure.        |
| smallsh         | Your shell implementation. Named like a real project, not 'shell-project'.         |
| mymalloc        | Heap allocator. Stands alone as a portfolio artifact.                              |
| mylang          | Your toy language — grows from Phase 5 through Phase 7. One repo, tagged releases. |
| mycc            | Your compiler — forked or evolved from mylang when backend work begins.            |
| myos-c          | Your C kernel. Clearly named.                                                      |
| myos-rust       | Your Rust kernel. Separate repo.                                                   |
| regex-engine    | Phase 4 automata project.                                                          |
| lambda-calculus | Phase 4 type theory project.                                                       |
| asm-exercises   | Assembly exercises from Track A.                                                   |

## Commit Discipline

- Commit small and often. A commit should represent one logical change, not a day's work.
- Commit messages are imperative mood, present tense: 'Add phi node elimination' not 'Added...'
- Every repo must have a README.md that explains what the project is, how to build it, and what you learned.
- Every repo must have a DEVLOG.md. Date-stamped entries. What broke, what you tried, what worked.
- Tag milestones: v0.1 when the basic structure works, v1.0 when the stated goals are met.
- Never push generated build artifacts. Maintain .gitignore from the start.
- Prefer many small repos over one monorepo. Each project should stand on its own.
- Add a GitHub Actions CI badge to every repo. Failing CI on main is not acceptable.

## README Template (every project)

- Project name and one-sentence description
- Build instructions (copy-paste ready, tested on a fresh machine)
- Usage examples with expected output
- Architecture overview (what are the major components and how do they fit)
- What I learned (honest reflection — this is the most valuable section for employers)
- Known limitations and future work
- References and resources used

---

*This roadmap ends where most people think it begins. The destination — a working compiler and a working kernel — are not end states. They are platforms from which to go further: into research compilers, into real OS contribution, into verified systems, into embedded targets. The goal of this document is simply to make those doors visible and the path to them concrete.*